

Sisteme de operare

Laborator 1

Semestrul I 2025

Prezentare generala laborator

- programare de sistem in Linux (clona Unix)
- “Advanced Programming in the Unix Environment”, W. Richard Stevens
- gestiunea fisierelor si I/O
- controlul proceselor (+threads)
- semnale
- IPC (inter-process communication)
 - semafoare System V, monitoare pt pthreads
 - shared memory System V vs POSIX
 - programare client-server (TCP/IP sockets, RPC)
- advanced topics

Laborator 1

- recapitulare concepte ITBI (perspectiva programarii de sistem)
 - procesul de login utilizator
 - interpretorul de comenzi
 - interfata cu nucleul (kernelul) sistemului de operare

Logarea utilizatorului

- *getty* afiseaza prompt-ul de login
- utilizatorul introduce numele de utilizator
- *getty* apeleaza programul */bin/login* cu numele de utilizator ca parametru
- *login*: stabileste o noua sesiune de lucru
 - afiseaza promptul pt parola
 - cauta in */etc/passwd* o intrare corespunzatoare numelui de utilizator
 - verifica parola (de regula stocata criptat in alt fisier, eg. */etc/shadow*)
 - daca parola e corecta
 - executa interpretorul de comenzi (*shell*-ul) asociat intrarii identificate
 - ii asociaza *shell*-ului variabile de mediu (environment) initializate cu valorile din campurile citite din intrarea corespunzatoare din */etc/passwd* (in mod notabil USER, SHELL, HOME)
- *shell*-ul afiseaza un prompt specific (\$) pt utilizator obisnuit) si asteapta comenzile utilizatorului
- *init* monitorizeaza sesiunea de lucru a utilizatorului si cand acesta incheie activitatea (paraseste *shell*-ul), reporneste o instanta a programului *getty* pe terminalul respectiv

Identificarea utilizatorului

- la login, utilizatorul primește un ID propriu (valoare întreaga nenegativă care identifică utilizatorul în sistemul de operare)
 - obținut din intrarea corespunzătoare utilizatorului din */etc/passwd*
 - în general unic
 - asignat de către administratorul sistemului, singurul care are permisiunea de a scrie în */etc/passwd*, și nu poate fi schimbat de către utilizator
 - folosit de kernel pentru a verifica dacă utilizatorul (mai exact procesele sale) are dreptul să execute anumite operații
- utilizatorul cu UID = 0 s.n. *root* sau *superuser*
 - intrarea corespunzătoare din */etc/passwd* folosește de regulă numele de *root*
- procesele *root* au privilegii de *superuser* și de regulă circumventează verificările pe care kernelul le face pentru o serie de operații
- unele dintre funcțiile kernelului pot fi executate doar de procese *root*
- *root*-ul are control total asupra sistemului de calcul
 - Obs: din acest motiv, este puternic descurajată inițiativa utilizatorilor sistemului care știu parola de *root* să ruleze programe obișnuite în calitate de *root* (UID = 0)

Identificarea utilizatorului (cont.)

- la login utilizatorul nu primește doar un UID ci și un GID (Group ID), setat tot de administrator în intrarea corespunzătoare din */etc/passwd*
 - GID-ul permite partajarea de resurse între membri ai aceluiași grup, chiar dacă au UID-uri diferite
 - de ex, intrările de director pentru fiecare fișier din sistem conțin perechea (UID,GID) a proprietarului fișierului respective
 - comanda shell *ls -l* permite afișarea ID-urilor proprietarului fișierului
 - în schimb, utilizatorii cu GID diferit nu pot accesa aceste resurse partajate ale grupului
- fișierul */etc/group* asignează nume lizibile GID-urilor utilizator;
 - modificabil doar de către administratorul sistemului
 - listează și *supplementary GIDs*, i.e. același utilizator poate avea mai multe GID-uri (poate aparține mai multor grupuri)
- comanda shell cu care se pot afla UID/GID : */usr/bin/id*

Interpretorul de comenzi, shell

- utilizabil deopotriva in mod interactiv cat si batch (folosind *shell script*-uri)
- ofera posibilitatea de a executa atat comenzi interne (executate in cadrul interpretorului) cat si externe (programe incarcate de pe disc)
- functionalitati principale
 - asigurarea unui mediu de lucru utilizatorului (v. comanda *env*)
 - comenzi de manipulare a fisierelor si directoarelor
 - comenzi de control al executiei programelor
 - controlul si monitorizarea activitatilor de I/O
 - administrarea sistemului (rezervata unui utilizator special cunoscut in mod uzual sub numele de *root*, cu $UID = 0$, v. prima intrare din */etc/passwd*)
 - *samd.*
- ex. interpretoare de comenzi: Bourne Shell (*/bin/sh*), Bourne Again Shell (*/bin/bash*), C Shell (*/bin/csh*), Korn Shell (*/bin/ksh*), etc.

/etc/shells contine interpretoarele de comenzi disponibile in sistem

The Bourne-Again SHell

- */bin/bash*, urmasul primului shell istoric, */bin/sh* (Bourne Shell)
- fisiere de configurare
 - fisiere de start-up inspectate doar la login
 - system-wide: */etc/profile*
 - locale, in home directory: *~/.profile*, *~/.bash_profile*, *~/.bash_login*
 - continutul lor e executat automat la login
 - fisiere de start-up inspectate la crearea fiecarui terminal (rc file, “run commands”)
 - *~/.bashrc*
 - continutul lor poate fi executa voluntar cu comanda *source* (sau “.”)

```
source .bashrc
```

```
..bashrc
```
 - fisiere de logout: *~/.bash_logout*
 - continutul este executat la iesirea din shell (cu *exit* sau *^D*)
 - Obs: ctrl-D in Unix este caracterul EOF, cand utilizatorul tipareste *^D* shell-ul se termina
- istoria comenzilor este inregistrata in *~/.bash_history*

Mediul de lucru (environment)

- o lista de perechi *nume = valoare*
- stocare de shell si disponibile programelor executate din shell
- comanda */usr/bin/env* afiseaza variabilele de mediu
- intr-un program C, variabilele de mediu sunt accesibile in cel de-al treilea parametru al functiei *main*

```
int main(int argc, char* argv[], char *envp[])
```

- cand shell-ul executa programul C, ii furnizeaza mediul de lucru in cel de-al treilea argument al functiei *main*

Descriptori de fișiere

- întregi nenegativi folosiți pentru identificarea fișierelor deschise în sistem
- când kernelul deschide un fișier existent sau creează unul nou ca urmare a solicitării unui proces întoarce un descriptor de fișier care va putea fi folosit de proces pentru citirea/scrierea fișierului
- descriptori speciali
 - la pornirea oricărui program, shell-ul deschide pentru acesta trei descriptori de fișiere speciali:
 - 0 standard input
 - 1 standard output
 - 2 standard error
 - uzual, cei trei descriptori de fișier sunt asociați cu terminalul de login (sau terminalul de lucru, într-un mediu grafic cu multiple X terminale)
 - pentru a afla care este terminalul asociat unui shell (în general, nu doar terminalul de login) se folosește comanda `/usr/bin/tty`

Apeluri sistem

- toate procesele din sistem (inclusive comenzile shell, interne sau externe) pot solicita serviciile kernelului prin intermediul apelurilor sistem (*system calls*)
 - intrari in codul kernel (nr limitat)
 - apelabile in mod uzual prin intermediul bibliotecii *libc* care este linkeditata automat (cel mai adesea dinamic) la orice program executabil
 - *libc* contine wrapper-e cu acelasi nume ca si apelurile de sistem
 - wrapper-ele executa o instructiune privilegiata de tip trap (*syscall* in MIPS) sau intrerupere software (*int 80h* in x86 Linux)
- dpdv programator, apelurile sistem pot fi asimilate unor simple apeluri de biblioteca
- dpdv dezvoltator de cod kernel diferenta e fundamentala
- ex: *printf/cout* apeleaza *write*, *malloc/new* apeleaza *brk/sbrk*, *rand*
 - *write*, *brk* sunt apelurile sistem (de fapt stub-uri in *libc*)
 - *printf/malloc* sunt apeluri de biblioteca

Tratarea erorilor

- diferenta notabila intre apelurile sistem si apelurile obisnuite de biblioteca dpdv al utilizatorului: *tratarea erorilor*
- apel sistem terminat cu eroare => valoare de return negativa si variabila *errno* setata pe o anumita valoare
 - o valoarea de return zero semnifica in general succes
- programatorul are acces la *errno* in C in mod uzual:

```
#include <errno.h>
```

```
extern int errno;
```

- preferabil, folosirea functiilor de biblioteca de tip *strerror* sau *perror*

```
ret_code = ...      // apel sistem
```

```
if (ret_code < 0)
```

```
    perror("apel sistem eronat"), exit(1); N.B.: 0 = succes, >0 = esec
```

Ex. apeluri sistem: read/write

- *ssize_t read(int fd, void *buf, size_t count);*
 - intoarce nr de octeti cititi (0 daca s-a ajuns la EOF) in *buf*
 - cand *fd* refera un terminal, se citeste cate o linie (eg, pana la `\n`)

ex: citire de la tastatura

```
char buf[256];  
...  
ret_code = read(0, buf, 256);
```

- *ssize_t write(int fd, void *buf, size_t count);*
 - intoace nr de octeti scrisi

ex: scriere pe ecran (terminal)

```
char buf[256];  
...  
ret_code = write(1, buf, 256);
```

Sisteme de operare

Laborator 1

Interpretorul de comenzi, procesul de login

1. Creati o structura de directoare pe care sa o folositi pentru programele de la laborator, dupa cum urmeaza:

```
$ cd                                # schimbati directorul in home directory
                                   # echivalent cu "cd $HOME"

$ pwd                               # afisati valoarea directorului curent
                                   # echivalent cu "echo $PWD"

$ mkdir -p SO/laborator/lab1 # creati directorul SO cu subdirectoarele sale

$ cd SO/laborator/lab1            # schimbati directorul
$ pwd
$ mkdir doc src bin               # creati subdirectoarele doc, src, bin in directorul curent
$ ls -l                           # afisati continutul directorului curent
```

Creati apoi un fisier *.myrc* (un fisier de tip *run commands*, asa cum e si fisierul *.bashrc* din home directory) in directorul SO care sa creeze alias-uri pt comenzile *rm* si *ls* si sa extinda variabila de mediu *PATH* dupa cum urmeaza:

```
$ cd ../../                        # reveniti in directorul SO
$ pwd
$ echo alias rm=\rm -i\ >> .myrc  # adaugati la continutul fisierului .myrc
                                   # comanda care stabileste ca stringul "rm"
                                   # este un shortcut pentru comanda "rm -i"

$ echo alias l=\ls -l\ >> .myrc   # similar pentru "ls -l" abreviat cu "l"

$ ls -la                          # flagul -a permite afisarea fisierelor al
                                   # caror nume incepe cu ., cum e si .myrc, care
                                   # altfel nu sunt vizibile la un simplu "ls"

$ cat .myrc                       # afiseaza la stdout continutul fisierului .myrc

$ env                             # afisati continutul variabilelor de mediu

$ env | grep PATH                 # string matching, grep selecteaza doar
                                   # continutul variabilei PATH dintre toate
                                   # variabilele de mediu afisate de env
                                   # comanda de tip pipeline, iesirea comenzii env
                                   # e redirectata catre intrarea comenzii grep

$ env | grep HOME                 # comenzile env si grep se lanseaza
                                   # concurrent/paralel si se sincronizeaza prin pipe
                                   # (canal de comunicatie unidirectional)

$ echo export PATH=$PATH:$HOME/SO/laborator/lab1/bin >> .myrc
```

```

# extinde continutul variabilei PATH cu
# stringul "$HOME/SO/laborator/lab1/bin"
# comanda interna export face variabila PATH
# disponibila tuturor programelor care se
# executa din shell
# orice executabil care se gaseste intr-un
# director din PATH poate fi executat din
# orice alt director fara a i se furniza calea
# completa
# executabilele care nu se gasesc intr-un director
# din PATH trebuie specificate cu cale completa
# pentru a putea fi executate

$ echo $PATH ; echo $HOME      # afisati valorile variabilelor PATH si HOME
                                # comanda compusa, lista de comenzi care se
                                # executa secvential

$ cat .myrc
$ source .myrc                 # executa toate comenzile stocate in .myrc
                                # si pastreaza in shell valorile modificate
                                # echivalent cu comanda ". .myrc"

$ rm .myrc                     # ce se intampla daca stergeti fisierul .myrc?
                                # dispare efectul comenzilor pe care le-ati
                                # executat mai sus cu comanda source ?

$ cd -                          # schimbati directorul catre directorul anterior
$ pwd
$ l -a                          # l este ls -l dupa source-ul de mai sus, iar
                                # flagul -a se adauga si comanda devine ls -la
                                # mai exact ls -l -a

```

Pentru a intelege mai bine ce face fiecare comanda, cititi paginile de manual (eg, *man mkdir*). In subdirectorul *doc* veti salva documentatia de laborator, in subdirectorul *src* veti crea programele sursa C (folosind editorul text preferat), iar in subdirectorul *bin* veti pune programele executabile rezultate ca urmare a compilarii programelor sursa C.

Compilarea programelor C se va face cu comanda *gcc*, compilatorul C, dupa cum urmeaza:

```

$ cd $HOME/SO/laborator/lab1/src      # acelasi lucru cu
                                        # cd ~/SO/laborator/lab1/src
                                        # ~ este echivalent cu home directory

$ pwd
$ gcc -o ../bin/fisier_executabil fisier_sursa.c
                                        # compileaza fisierul sursa C din directorul
                                        # curent, i.e. lab1/src, si plaseaza executabilul
                                        # nou creat in directorul lab1/bin

```

unde *fisier_executabil* si *fisier_sursa* sunt nume de fisiere pe care le alegeti dupa dorinta fiecaruia.

Apoi puteti executa programul executabil rezultat, din orice director, cu comanda

```
$ fisier_executabil
```

De ce e posibil asa ceva? In fond, *fișier_executabil* se afla in subdirectorul *lab1/bin* si in principiu ca sa poate fi executat trebuie furnizata calea completa a executabilului sau trebuie schimbat directorul in *lab1/bin* inainte de a executa programul. Compilati programul urmator, de la punctul 2, si vedeti ce se intampla.

2. Scrieti un program C *pwnam.c* in directorul *lab1/src* care primeste ca parametru in linia de comanda un nume de utilizator si afiseaza informatii despre utilizator cum ar fi: user si group ID-ul, home directory, shell. Compilati executabilul cu numele *pwnam* in directorul *lab1/bin* cf. comenzilor de mai sus si executati programul:

```
$ gcc -o ../bin/pwnam pwnam.c
$ pwnam
```

Dupa compilarea si executarea programului, stergeti executabilul si apelati comanda *gcc* fara flag-ul *-o*, ca mai jos:

```
$ rm ../bin/pwnam
$ pwnam
$ gcc pwnam.c
$ l # sau ls -l daca nu va functioneaza alias-ul de mai sus
```

Cine e fisierul *a.out*? Ce se intampla daca il executati ca mai jos?

```
$ ./a.out
```

Dar daca incercati sa-l executati astfel:

```
$ a.out
```

De ce nu mai functioneaza? Ce legatura are cu variabila de mediu *PATH*?

Cum puteti sa recreati comanda *pwnam* (rezultatul compilarii de mai sus) folosind fisierul *a.out* , fara sa recompilati programul *pwnam.c*?

Indicatie: Folositi functia *getpwnam* (*man getpwnam*). Numele de utilizator il puteti lua din fisierul */etc/passwd* pe care il puteti vizualiza cu comenzile de tip pager *more/less* (*man more*, *man less*).

3. Scrieti un program C *getids.c* in directorul *lab1/src* care afiseaza PID-ul, UID-ul si GID-ul procesului curent. Nu uitati ca desi este doar un program, el apartine unui utilizator (cel care il ruleaza) si atunci pe langa PID are si UID si GID. Compilati executabilul cu numele *getids* in directorul *lab1/bin* cf. comenzilor de mai sus.

Indicatie: Folositi functiile *getpid*, *getuid*, *getgid*. Folositi comanda *man* pentru a afla cum sa le utilizati.

4. Scrieti un program C care emuleaza comanda *env* (*man env*). Programul poate primi ca parametru numele unei variabile de mediu (eg, *HOME*) si tipareste pe ecran continutul variabilei respective.

Indicatie: Parcurgeti vectorul de string-uri *envp* (al treilea argument de apel al functiei *main*) in cautarea numelui variabilei de mediu. Pentru comparatia de string-uri folositi *strcmp/strncmp*.

5. Scrieti un program C care copiaza in bucla informatia citita de la standard input (file descriptor 0) la standard output (file descriptor 1). Cum puteti folosi acest program pentru a simula efectul comenzii *cp*, care copiaza un fisier sursa intr-un fisier destinatie?